



Kotlin Multiplatform

Condividere codice tra Android, iOS, Desktop e Web

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Cos'è Kotlin Multiplatform

Kotlin Multiplatform (KMP) è una tecnologia di JetBrains che permette di scrivere logica di business una sola volta in Kotlin e compilarla per più piattaforme: Android, iOS, Desktop, Web e Server.

- Il codice condiviso gira nativamente su ogni piattaforma (JVM, Native, JS/WASM).
- Non è un framework UI cross-platform: la UI può restare nativa o usare Compose Multiplatform.
- Condividi la logica di business, networking, modelli, validazione, persistenza.
- Ogni piattaforma mantiene accesso completo alle API native (nessun bridge).
- Stabile da Kotlin 1.9.20 (novembre 2023), raccomandato da Google per Android+iOS.
- Integrato in Android Studio e Fleet; supportato anche da Xcode (via framework).



Google ha adottato ufficialmente KMP come approccio raccomandato per condividere logica tra Android e iOS.

Target supportati

KMP compila Kotlin in codice nativo per ciascuna piattaforma. Non c'è un runtime intermedio.



Android

Compila in JVM bytecode. Usa Gradle, Android Studio, tutte le API Android.



iOS

Compila in codice nativo ARM via Kotlin/Native. Genera un .framework per Xcode.



Desktop

JVM per Windows, macOS, Linux. Con Compose Desktop per la UI.



Web

Compila in JavaScript o WebAssembly. Compatibile con React, node.js.

Prerequisiti

Prima di creare un progetto KMP assicurati di avere installato e aggiornato tutti gli strumenti necessari.



Android Studio Hedgehog+ (2023.1+)

L'IDE principale. Le versioni recenti includono il supporto KMP integrato. Scarica da developer.android.com/studio.



Plugin Kotlin Multiplatform Mobile

Vai in Settings → Plugins → Marketplace, cerca "Kotlin Multiplatform" e installa il plugin ufficiale di JetBrains.



Xcode 15+ (solo su macOS)

Necessario per compilare il target iOS. Installalo dal Mac App Store. Accetta la licenza con: `sudo xcodebuild -license accept`.



CocoaPods o Xcode CLI tools

Per integrare il framework KMP in Xcode: `brew install cocoapods` oppure Xcode → Preferences → Locations → Command Line Tools.



Alternativa: KMP Wizard online

Puoi anche usare kmp.jetbrains.com per generare il progetto nel browser e scaricarlo come ZIP, senza bisogno del plugin.

Metodo 1: New Project in Android Studio

Android Studio con il plugin KMP offre un template diretto nella schermata di creazione progetto.

1

File → New → New Project

Nella schermata dei template seleziona "Kotlin Multiplatform App" nella sezione "Phone and Tablet".

2

Configura il nome e il package

Name: "MyKmpApp", Package: "com.example.mykmpapp", Minimum SDK: API 24+, Save location: scegli la cartella.

3

Scegli i target e la UI

Seleziona: Android ✓, iOS ✓. Per la UI condivisa scegli "Share UI (with Compose Multiplatform)" o "Do not share UI".

4

Finish → Gradle Sync

Android Studio genera il progetto, scarica le dipendenze e sincronizza Gradle. La prima sync può richiedere qualche minuto.

Metodo 2: KMP Wizard online

Il KMP Wizard di JetBrains è uno strumento web che genera il progetto senza bisogno del plugin.

1

Apri kmp.jetbrains.com

Nel browser, vai su kmp.jetbrains.com. Ti si presenta un configuratore visuale.

2

Configura il progetto

Nome, package, target (Android/iOS/Desktop/Web), UI condivisa o nativa.

3

Scarica il progetto

Clicca "Download" → ottieni un file ZIP con la struttura completa del progetto.

4

Apri in Android Studio

Decomprimi il ZIP, poi: File → Open → seleziona la cartella → Gradle sync automatico.

Opzioni del Wizard

Opzione	Scelte disponibili
Target piattaforme	Android, iOS, Desktop (JVM), Web (WASM), Server
UI condivisa	Compose Multiplatform (condivisa) oppure UI nativa per piattaforma
Librerie incluse	Ktor, kotlinx.serialization, Room, SQLDelight (opzionali)
Testing	kotlin.test incluso per commonTest

Struttura di un progetto KMP

Un progetto KMP è organizzato in source set: `commonMain` per il codice condiviso, `androidMain` e `iosMain` per il codice specifico.

```
shared/  
  src/  
    commonMain/kotlin/ ← codice condiviso  
    commonTest/kotlin/ ← test condivisi  
    androidMain/kotlin/ ← implementazioni Android  
    iosMain/kotlin/ ← implementazioni iOS  
  build.gradle.kts  
  androidApp/ ← app Android (Compose)  
    src/main/  
  iosApp/ ← progetto Xcode (SwiftUI)  
  iosApp.xcodeproj
```

Source set chiave

`commonMain`

Codice condiviso: modelli, logica, interfacce. Compilato per tutti i target.

`androidMain`

Implementazioni Android: actual class, accesso a Context, API Android.

`iosMain`

Implementazioni iOS: actual class con Foundation/UIKit, codice nativo Swift-interop.

`commonTest`

Test condivisi con kotlin.test. Girano su tutti i target automaticamente.

Configurazione Gradle

Il modulo shared dichiara i target e le dipendenze per ogni source set.

```
// shared/build.gradle.kts
plugins {
    kotlin("multiplatform")
    kotlin("plugin.serialization")
    id("com.android.library")
}

kotlin {
    androidTarget()
    iosX64(); iosArm64(); iosSimulatorArm64()

    sourceSets {
        val commonMain by getting {
            dependencies {
                implementation("io.ktor:ktor-client-core:2.3.7")
                implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.2")
            }
        }
        val androidMain by getting {
            dependencies { implementation("io.ktor:ktor-client-okhttp:2.3.7") }
        }
        val iosMain by getting {
            dependencies { implementation("io.ktor:ktor-client-darwin:2.3.7") }
        }
    }
}
```

Kotlin

expect / actual: il meccanismo chiave

expect dichiara un'API nel codice condiviso. **actual** la implementa in ogni piattaforma. Il compilatore verifica la corrispondenza.

commonMain (dichiarazione)

```
// commonMain/kotlin/Platform.kt
expect fun getPlatformName(): String
expect class DeviceInfo() { fun osVersion(): String }
```

Kotlin

androidMain (implementazione Android)

```
// androidMain
actual fun getPlatformName() = "Android"

actual class DeviceInfo {
    actual fun osVersion() =
        "Android ${Build.VERSION.SDK_INT}"
}
```

Kotlin

iosMain (implementazione iOS)

```
// iosMain
actual fun getPlatformName() = "iOS"

actual class DeviceInfo {
    actual fun osVersion() =
        UIDevice.current.systemVersion
}
```

Kotlin



Il compilatore garantisce che ogni **expect** abbia un **actual** corrispondente in tutti i target.

expect/actual: casi d'uso comuni

expect/actual si usa per qualunque cosa che dipende dalla piattaforma: database, filesystem, UUID, logging.

Esempio: generare un UUID

```
// commonMain Kotlin
expect fun randomUUID(): String

// androidMain
actual fun randomUUID(): String =
    java.util.UUID.randomUUID().toString()

// iosMain
actual fun randomUUID(): String =
    NSUUID().UUIDString
```

Esempio: logging per piattaforma

```
// commonMain Kotlin
expect fun log(tag: String, msg: String)

// androidMain
actual fun log(tag: String, msg: String) =
    android.util.Log.d(tag, msg)

// iosMain
actual fun log(tag: String, msg: String) =
    NSLog("[tag] $msg")
```

Alternativa moderna: interfaccia + DI

```
// commonMain - interfaccia Kotlin
interface Logger { fun d(tag: String, msg: String) }

// androidMain - implementazione iniettata con Koin / manual DI
class AndroidLogger : Logger {
    override fun d(tag: String, msg: String) = Log.d(tag, msg)
}
```

Ecosistema di librerie KMP

L'ecosistema KMP è maturo e in rapida crescita. Ecco le librerie chiave per la logica condivisa.

Ktor Client

Client HTTP multiplatform. Engine OkHttp (Android) e Darwin (iOS).
Suspend fun native.

kotlinx.serialization

Serializzazione JSON/Protobuf/CBOR in codice comune. Annotazioni
@Serializable.

SQLDelight

Database SQL type-safe. Genera codice Kotlin dalle query .sq. Driver per
ogni piattaforma.

kotlinx.datetime

Date, time, timezone multiplatform. Sostituisce java.time / NSDate.

Koin

Dependency injection leggera e multiplatform. Moduli condivisi +
moduli per piattaforma.

Compose Multiplatform

UI condivisa con Jetpack Compose su Android, iOS, Desktop e Web
(alpha).

Compose Multiplatform

Compose Multiplatform estende Jetpack Compose di Android a iOS, Desktop e Web. Scrivi la UI una volta, compila per tutti i target.

```
// commonMain - UI condivisa
@Composable
fun App() {
    MaterialTheme {
        var name by remember { mutableStateOf("") }
        Column(
            Modifier.fillMaxSize().padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                "Ciao da ${getPlatformName()}!",
                style = MaterialTheme.typography.headlineMedium
            )
            Spacer(Modifier.height(16.dp))
            OutlinedTextField(
                value = name,
                onValueChange = { name = it },
                label = { Text("Il tuo nome") }
            )
            Spacer(Modifier.height(8.dp))
            Button(onClick = { }) {
                Text("Benvenuto, $name!")
            }
        }
    }
}
```

Kotlin

Stato della UI condivisa

Android

Stabile. Stesso Jetpack Compose.

iOS

Stabile (1.6+). Rendering via Skiko.

Desktop

Stabile. JVM + Skiko.

Web (WASM)

Alpha. In rapida evoluzione.

Entry point per ogni piattaforma

Ogni piattaforma ha un entry point che avvia il composabile condiviso App().

Android (MainActivity.kt)

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(b: Bundle?) {  
        super.onCreate(b)  
        setContent { App() } // composabile condiviso  
    }  
}
```

Kotlin

iOS (iosApp/ContentView.swift)

```
import shared // framework KMP generato  
  
struct ContentView: View {  
    var body: some View {  
        ComposeView()  
        .ignoresSafeArea()  
    }  
}
```

Swift

Desktop (main.kt)

```
fun main() = application {  
    Window(title = "KMP App") {  
        App() // stesso composabile  
    }  
}
```

Kotlin

Web (WASM)

```
@OptIn(ExperimentalComposeUiApi::class)  
fun main() {  
    CanvasBasedWindow("KMP App") {  
        App() // stesso composabile  
    }  
}
```

Kotlin

Networking con Ktor

Ktor Client è la libreria HTTP multiplatform ufficiale di JetBrains. Usa coroutine e suspend fun.

```
// commonMain
@Serializable
data class User(val id: Int, val name: String, val email: String)

class UserApi(private val client: HttpClient) {

    suspend fun getUsers(): List<User> =
        client.get("https://api.example.com/users").body()

    suspend fun getUser(id: Int): User =
        client.get("https://api.example.com/users/$id").body()

    suspend fun createUser(user: User): User =
        client.post("https://api.example.com/users") {
            contentType(ContentType.Application.Json)
            setBody(user)
        }.body()
}

// Factory del client (commonMain)
fun createHttpClient() = HttpClient {
    install(ContentNegotiation) { json() }
    install(Logging) { level = LogLevel.BODY }
}
```

Kotlin

Ktor: engine per piattaforma

L'API è identica in commonMain. L'engine cambia per piattaforma: OkHttp su Android, Darwin su iOS.

commonMain (interfaccia condivisa)

```
// commonMain - expect per il client
expect fun createPlatformHttpClient(): HttpClient
```

Kotlin

androidMain

```
// androidMain
actual fun createPlatformHttpClient() =
    HttpClient(OkHttp) {
        install(ContentNegotiation) {
            json()
        }
    }
```

Kotlin

iosMain

```
// iosMain
actual fun createPlatformHttpClient() =
    HttpClient(Darwin) {
        install(ContentNegotiation) {
            json()
        }
    }
```

Kotlin



Alternativa: CIO engine

CIO (Coroutine I/O) è un engine Kotlin puro che funziona su tutti i target, ma ha meno feature di OkHttp/Darwin.

Persistenza con SQLDelight

SQLDelight genera codice Kotlin type-safe dalle query SQL. Supporta Android (SQLite) e iOS (nativo).

1. Definisci le query (.sq)

```
-- shared/src/commonMain/sqlDelight/User.sql      SQL
CREATE TABLE User (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL
);

selectAll: SELECT * FROM User;
insertUser: INSERT INTO User VALUES (?, ?, ?);
deleteById: DELETE FROM User WHERE id = ?;
```

2. Usa il codice generato

```
// commonMain - repository                          Kotlin
class UserRepository(db: AppDatabase) {
  private val q = db.userQueries

  fun getAll(): List<User> =
    q.selectAll().executeAsList()

  fun insert(id: Long, name: String, email: String) =
    q.insertUser(id, name, email)

  fun delete(id: Long) =
    q.deleteById(id)
}
```

```
// androidMain: driver = AndroidSqliteDriver(AppDatabase.Schema, context, "app.db")
// iosMain:      driver = NativeSqliteDriver(AppDatabase.Schema, "app.db")
expect fun createDriver(): SqlDriver // pattern expect/actual      Kotlin
```

Testing condiviso

I test in `commonTest` girano automaticamente su tutti i target. Usa `kotlin.test` per le asserzioni.

```
// commonTest
class UserApiTest {

    @Test
    fun platformNameNotEmpty() {
        val name = getPlatformName()
        assertTrue(name.isNotEmpty())
    }

    @Test
    fun userSerializationRoundTrip() {
        val user = User(1, "Alice", "a@b.com")
        val json = Json.encodeToString(user)
        val decoded = Json.decodeFromString<User>(json)
        assertEquals(user, decoded)
    }

    @Test
    fun uuidIsUnique() {
        val a = randomUUID()
        val b = randomUUID()
        assertNotEquals(a, b)
    }
}
```

Kotlin

Framework di test

- `kotlin.test`: asserzioni cross-platform
- `@Test`, `assertEquals`, `assertTrue`, `assertFails`
- `commonTest`: compilato per tutti i target
- `androidTest`: test specifici Android
- `iosTest`: test specifici iOS
- Ktor `MockEngine` per test di rete

KMP vs Flutter vs React Native

Ogni approccio ha trade-off diversi. KMP è l'unico che condivide logica mantenendo UI native.

	KMP	Flutter	React Native
Linguaggio	Kotlin	Dart	JavaScript/TypeScript
Cosa condivide	Logica (+ UI opzionale)	Logica + UI	Logica + UI
UI nativa	Sì (o Compose MP)	No (propri widget)	Parziale (bridge)
Rendering	Nativo per piattaforma	Skia engine proprio	Bridge a componenti nativi
Performance	Nativa	Quasi nativa	Bridge overhead
Accesso API native	Diretto (expect/actual)	Plugin/FFI	Bridge / Turbo Modules
Ecosistema Android	Completo (è Kotlin!)	Separato	Separato
Curva apprendimento	Bassa (se sai Kotlin)	Media (Dart)	Bassa (se sai JS)

Eseguire su Android

L'app Android si esegue come qualunque altro progetto Android: seleziona il modulo `androidApp` e premi Run.

1

Seleziona la configurazione "androidApp"

Nel menu a tendina delle configurazioni (in alto a sinistra), scegli "androidApp". Se non c'è, fai Edit Configurations → Add → Android App.

2

Scegli il dispositivo

Seleziona un emulatore AVD o un device fisico collegato via USB con debug USB abilitato.

3

Premi Run ► (Shift+F10)

Gradle compila il modulo `:shared` (per JVM), poi `:androidApp`. L'APK viene installato e l'app si avvia.

i

Prima build lenta

La prima compilazione scarica le dipendenze e configura i target. Le build successive sono molto più veloci grazie alla cache Gradle.

Eseguire su iOS

Per iOS servono macOS e Xcode. Ci sono due modi per eseguire l'app.

Metodo A: da Android Studio (con plugin KMP)

1

Seleziona "iosApp"

Nel menu delle configurazioni, scegli "iosApp" (creata dal plugin KMP).

2

Scegli il simulatore iOS

Seleziona un simulatore iPhone/iPad e premi Run ►.

Metodo B: da Xcode (per debug avanzato)

1

Apri iosApp/ in Xcode

Doppio click su iosApp.xcodeproj o .xcworkspace (se usi CocoaPods).

2

Build & Run

Xcode compila il framework Swift e il modulo shared Kotlin/Native, poi avvia il simulatore.

!

Solo su macOS

La compilazione iOS (Kotlin/Native) funziona esclusivamente su macOS con Xcode installato. Su Windows/Linux puoi sviluppare solo il target Android.

i

Dopo ogni modifica al modulo :shared, ricordati di ri-compilare il framework prima di runnare in Xcode.

Quando scegliere KMP

KMP eccelle quando vuoi condividere logica mantenendo pieno controllo sulla UI nativa di ciascuna piattaforma.

IDEALE PER

- Il team conosce già Kotlin/Android
- Serve UI nativa (pixel-perfect)
- App con molta logica di business
- Integrazione profonda con le API native
- Progetto Android esistente da espandere a iOS
- Performance critica (nessun bridge)

MENO ADATTO SE

- Il team è 100% web/JS → React Native
- Serve prototipo rapidissimo con UI identica
- App semplice senza logica complessa
- Nessuno nel team conosce Kotlin
- Serve supporto Web maturo oggi
- Budget non copre due UI native

Best practice KMP

DA FARE

- Massimizza il codice in commonMain
- Usa librerie KMP-native (Ktor, SQLDelight)
- Interfacce + DI per le dipendenze native
- Test condivisi in commonTest
- expect/actual solo per API di piattaforma
- Usa il KMP Wizard per nuovi progetti
- Aggiorna Kotlin/KMP regolarmente

DA EVITARE

- Mettere logica UI nel modulo shared
- Usare librerie JVM-only in commonMain
- expect/actual per ogni piccola cosa
- Ignorare le differenze di threading iOS
- Duplicare logica nelle app native
- Dimenticare di testare su iOS reale
- Mescolare Gradle Groovy e KTS

Errori comuni e soluzioni

InvalidMutabilityException (iOS)

Kotlin/Native ha un modello di memoria diverso. Da K/N 1.7+ il new memory manager risolve quasi tutto.

Cocoapods non trovato

Configurare cocoapods {} nel build.gradle o usa SPM (Swift Package Manager) per integrare il framework.

Framework non aggiornato in Xcode

Dopo modifiche al codice shared: pulire la build, rigenerare il framework e ri-buildare in Xcode.

Libreria JVM-only in commonMain

Non compilare per iOS. Cercare l'equivalente KMP o wrappare con expect/actual.

Build lenta su iOS

La compilazione Kotlin/Native è più lenta. Usare il cache di compilazione e compila solo il target necessario.

Crash su thread iOS

Accedere a oggetti Kotlin da thread diversi su iOS poteva causare crash. Risolto col nuovo MM (default da 1.7.20).

Cheat-sheet KMP

Kotlin

```
// 1. expect/actual
expect fun getPlatformName(): String    // commonMain
actual fun getPlatformName() = "Android" // androidMain

// 2. Ktor Client
val client = HttpClient { install(ContentNegotiation) { json() } }
val users: List<User> = client.get("https://api.example.com/users").body()

// 3. kotlinx.serialization
@Serializable data class User(val id: Int, val name: String)
val json = Json.encodeToString(user)
val user = Json.decodeFromString<User>(json)

// 4. SQLDelight
val users = db.userQueries.selectAll().executeAsList()
db.userQueries.insertUser(1, "Alice", "alice@mail.com")

// 5. Compose Multiplatform
@Composable fun App() { MaterialTheme { Text(getPlatformName()) } }

// 6. Test condiviso (commonTest)
@Test fun testPlatform() { assertTrue(getPlatformName().isNotEmpty()) }

// 7. Dipendenze per piattaforma
// commonMain: ktor-client-core      androidMain: ktor-client-okhttp
// iosMain: ktor-client-darwin        desktopMain: ktor-client-cio
```